iShop en la nube: cómo se publicó y cómo se sostiene

Infraestructura, decisiones técnicas y operación del asistente virtual

Proyecto de tesis iShop

22 de octubre de 2026

Índice

1. Para qué sirve este documento 1
2. Qué se logró 2
   - Direcciones del sistema 2
   - Resultado medido 2
3. Las piezas del sistema y por qué cada una 2
   - Por qué no es simplemente "ChatGPT conectado a la tienda" 3
4. Cómo viaja un mensaje, paso a paso 3
5. Dónde vive el proyecto ahora 4
   - El servidor 4
   - El nombre de dominio 4
   - La seguridad 4
6. Cómo se hizo la publicación 5
   - Dos problemas reales que se resolvieron 5
7. Uso diario 6
   - Atender como asesor humano 6
   - Actualizar los productos 7
   - Respaldar las conversaciones 7
8. Cuánto cuesta y qué pasa después 7
9. Preguntas probables en la defensa 8
10. Limitaciones reconocidas 8
11. Glosario 9

1. Para qué sirve este documento

La guía anterior (`GUIA-PAUL-ISHOP.md`) explica cómo levantar el proyecto en una computadora personal. Este documento explica lo siguiente: **el mismo proyecto ahora vive en un servidor de internet**, con dirección pública y candado de seguridad, de modo que cualquiera puede entrar desde su celular sin que la computadora del autor esté encendida.

Está escrito para alguien con formación técnica que no programa. No hay que escribir código en ningún momento: los comandos se copian y se pegan en una ventana de Terminal.

2. Qué se logró

Antes	Ahora
Solo funcionaba en una laptop encendida	Funciona 24 horas en un servidor
Dirección `http://localhost:8080`	Dirección pública con HTTPS
Solo el autor podía probarlo	Cualquier persona con el enlace
Sin cifrado	Certificado de seguridad válido y automático

Direcciones del sistema

Qué es	Dirección
La tienda con el asistente	`https://100-52-28-14.sslip.io`
Bandeja del asesor humano	`https://chat.100-52-28-14.sslip.io`
Panel de automatizaciones	`https://n8n.100-52-28-14.sslip.io`

Resultado medido

La prueba automática (`chatbot/scripts/test_chatbot_e2e.py`) ejecutada sobre el servidor en producción dio **13 respuestas correctas de 13**, con un tiempo promedio de **1,27 segundos** por respuesta. El criterio mínimo del proyecto era 32 de 40 respuestas correctas.

3. Las piezas del sistema y por qué cada una

El sistema no es un solo programa, sino siete piezas que se reparten el trabajo. Cada una corre aislada dentro de un contenedor Docker, que se puede entender como una caja sellada con todo lo que ese programa necesita.

Pieza	Qué hace	Por qué se eligió
Caddy	Recibe todas las visitas, entrega el sitio y reparte el tráfico	Consigue y renueva el certificado HTTPS solo, sin configuración
Chatwoot	El chat que ve el cliente y la bandeja del asesor	Software libre, permite que un humano retome la conversación
n8n	El cerebro que coordina los pasos de cada respuesta	Los pasos se ven como un diagrama, no como código
Qdrant	Guarda el catálogo convertido en números para buscarlo por significado	Base de datos vectorial libre y liviana
Ollama	Convierte textos en números (vectores) dentro del servidor	No cuesta nada ni envía datos a terceros
Google Gemini	Entiende la intención y redacta la respuesta final	Tiene una cuota gratuita suficiente
PostgreSQL y Redis	Guardan las conversaciones y las tareas pendientes de Chatwoot	Son los que Chatwoot exige

Por qué no es simplemente "ChatGPT conectado a la tienda"

La diferencia central del proyecto es la técnica **RAG** (generación aumentada por recuperación). El modelo de lenguaje nunca inventa precios: primero se busca la evidencia en el catálogo propio y solo después se le pide al modelo que redacte **usando únicamente esa evidencia**.

En la práctica, el catálogo se partió en **219 documentos independientes**, uno por producto o por tema de información. Cada uno se convirtió en una lista de 768 números que representa su significado. Cuando alguien pregunta, su pregunta se convierte a números del mismo modo y Qdrant devuelve los documentos más cercanos. Esto evita el error típico de mezclar una MacBook con un cable o un adaptador en la misma respuesta.

4. Cómo viaja un mensaje, paso a paso

```
El cliente escribe en el widget de la tienda
  → Chatwoot recibe el mensaje y lo deja en estado "pendiente"
  → Chatwoot avisa a n8n (AgentBot)
      → Gemini interpreta qué está pidiendo realmente
      → Ollama convierte esa petición en números
      → Qdrant busca los productos más parecidos
      → n8n ordena los resultados, filtra por presupuesto y arma la evidencia
      → Gemini redacta la respuesta usando solo esa evidencia
  → La respuesta aparece en el mismo chat del cliente
```

Si el cliente pide un asesor (o escribe palabras como soporte, persona, humano, reclamo o queja), la conversación cambia a estado “abierto”, la inteligencia artificial deja de responder y el caso aparece en la bandeja de Chatwoot para que lo atienda una persona. Para devolverlo al asistente basta con regresar la conversación al estado “pendiente”.

Si Gemini se queda sin cuota gratuita, el sistema no se cae: una interpretación de respaldo recorre los mismos pasos de Ollama y Qdrant y entrega una respuesta verificada, aunque menos conversacional.

5. Dónde vive el proyecto ahora

El servidor

Característica	Valor
Proveedor	Amazon Web Services, servicio Lightsail
Ubicación	Virginia, Estados Unidos (la región más cercana con buena latencia)
Sistema operativo	Ubuntu 24.04 LTS
Memoria	4 GB de RAM más 2 GB de memoria de intercambio
Procesador	2 núcleos compartidos
Disco	80 GB
Dirección IP fija	100.52.28.14
Costo	24 dólares al mes, descontados de créditos gratuitos

El nombre de dominio

No se compró dominio. Se usa **sslip.io**, un servicio público y gratuito que traduce cualquier nombre que contenga la dirección IP hacia esa misma dirección. Por eso `100-52-28-14.sslip.io` funciona sin registrar nada, y los nombres `chat.` y `n8n.` que van delante también.

Si en el futuro se compra un dominio propio (por ejemplo `ishop-tesis.com`), basta con apuntarlo a la misma IP y volver a ejecutar un comando de configuración; no hay que rehacer nada.

La seguridad

- **Cifrado**: el certificado lo emite Let’s Encrypt, una autoridad reconocida, y Caddy lo renueva solo antes de que venza.
- **Puertas abiertas**: en el cortafuegos solo están abiertos el puerto 22 (administración remota), el 80 y el 443 (web). Todo lo demás está cerrado.
- **Servicios internos**: Qdrant, Ollama, PostgreSQL y Redis solo responden dentro del propio servidor; desde internet son invisibles.

- **Archivos privados**: el servidor publica la carpeta del proyecto para mostrar la tienda, así que se configuró una regla que devuelve "no encontrado" ante cualquier intento de leer contraseñas, scripts o archivos de configuración. Está comprobado: pedir `/chatbot/.env` devuelve error 404.
- **Registro cerrado**: después de crear el usuario administrador se desactivó la opción de que cualquiera abra una cuenta en el panel de Chatwoot.

6. Cómo se hizo la publicación

El proceso completo, en el orden en que ocurrió:

1. **Se creó la cuenta de AWS** en el plan gratuito, que entrega 100 dólares en créditos al registrarse y hasta 100 más por completar actividades guiadas.
2. **Se creó el servidor** con un comando, no a mano: se eligió el plan de 4 GB, se le asignó una dirección IP fija y se abrieron únicamente los tres puertos necesarios.
3. **Se copió el proyecto** desde la computadora al servidor con `rsync`, una herramienta de copia que solo transfiere lo que cambió. Se excluyeron a propósito los archivos de configuración locales, porque las contraseñas de la laptop no sirven en el servidor y habrían impedido una instalación limpia.
4. **Se preparó el servidor** instalando Docker y creando 2 GB de memoria de intercambio, un colchón para los momentos de arranque en que todos los programas piden memoria a la vez.
5. **Se generó la configuración de producción**, que reemplaza las direcciones `localhost` por las direcciones públicas y crea contraseñas nuevas al azar.
6. **Se levantaron los siete servicios** con un solo comando. La primera vez tarda cerca de una hora porque hay que descargar unos 12 GB de programas.
7. **Se configuró Chatwoot**: usuario administrador, buzón de la tienda, bot asociado y generación del identificador público que usa el widget.
8. **Se configuró n8n**: usuario administrador, importación de los cuatro flujos de trabajo y de las credenciales de Ollama y Qdrant, y activación del flujo principal.
9. **Se cargó el catálogo** en la base vectorial: 219 documentos.
10. **Se verificó todo** con la prueba automática y con una conversación real desde el navegador.

Dos problemas reales que se resolvieron

Vale la pena documentarlos porque son parte del aprendizaje del proyecto.

El procesador se quedó sin crédito de velocidad. El servidor contratado es de tipo "ráfaga": permite trabajar rápido durante un rato y después baja a una velocidad base. La instalación consumió todo ese margen, y convertir los 219 productos en vectores iba a tardar más de una hora. La solución fue calcular esos vectores en la computadora del autor y escribirlos directamente en la base del servidor a través de un túnel seguro. Tardó cinco minutos. Esto fue posible porque los scripts leen la dirección de cada servicio desde una variable de entorno en lugar de tenerla fija en el código.

Una credencial mal nombrada. La conexión a Qdrant se configuró con el campo `url`, pero n8n espera que se llame `qdrantUrl`. El resultado era que el asistente saludaba bien pero devolvía respuestas vacías ante cualquier consulta de catálogo. Se detectó leyendo el registro de errores del

servidor, que decía literalmente “Invalid Qdrant URL”. Tras corregirlo, la prueba pasó de 3 aciertos a 13 de 13.

7. Uso diario

Todo se maneja desde la aplicación **Terminal**. Primero hay que entrar al servidor:

```
ssh -i ~/.ssh/ishop-lightsail.pem ubuntu@100.52.28.14
```

Una vez dentro, estos son los comandos de uso frecuente:

```
cd ~/ishop/chatbot

docker compose ps            # ver si los siete servicios están encendidos
docker compose logs -f n8n   # ver qué está pasando (salir con Control+C)
docker compose restart n8n   # reiniciar un servicio puntual
docker compose down          # apagar todo sin perder datos
free -h                      # ver cuánta memoria queda libre
```

Para volver a encender todo:

```
cd ~/ishop
bash chatbot/deploy/start-prod.sh
```

Para salir del servidor se escribe `exit`.

Atender como asesor humano

El asesor no usa una herramienta aparte: trabaja dentro de Chatwoot, en `https://chat.100-52-28-14.sslip.io`.

Lo primero al entrar es marcarse como **Disponible** en el menú de la foto de perfil, arriba a la derecha. Mientras nadie esté disponible, el widget de la tienda muestra el aviso “Estamos ausentes”.

Las conversaciones se ven en **Conversaciones → iShop Web**, y el estado indica quién está a cargo:

Estado	Significado
Pendiente	La inteligencia artificial está respondiendo
Abierta	El cliente pidió un asesor y espera respuesta humana
Resuelta	La consulta terminó

Cuando alguien escribe “quiero hablar con un asesor” (también funciona con soporte, persona, humano, agente, reclamo o queja), pasan tres cosas sin intervención: la conversación cambia a Abierta, se asigna al asesor configurado en `CHATWOOT_HUMAN_AGENT_EMAIL` y la inteligencia artificial deja de responder para no pisar al humano. Además queda una nota interna, visible solo para el equipo, indicando que hubo un traspaso.

El asesor responde desde el cuadro de texto normal y el cliente lo lee en el mismo widget, sin cambiar de ventana ni recargar la página. Para devolver la conversación a la inteligencia artificial, basta cambiar el estado a Pendiente.

Una advertencia sobre el alta de asesores: la pantalla de configuración inicial de Chatwoot crea una cuenta nueva cada vez que se completa. Un asesor que la use en lugar de iniciar sesión termina en una cuenta vacía, sin conversaciones. Los asesores adicionales se agregan desde **Settings → Agents** de la cuenta ya existente, y reciben una invitación por correo.

Actualizar los productos

Cuando cambien precios o productos, el procedimiento es:

```
cd ~/ishop
python3 chatbot/scripts/sync_knowledge.py    # recoge la información de la tienda
python3 chatbot/scripts/index_knowledge.py   # rehace la base vectorial
```

El segundo comando puede tardar bastante en el servidor por el límite de velocidad del procesador. Si urge, se puede hacer desde la computadora personal como se explicó en el apartado anterior.

Respaldar las conversaciones

```
cd ~/ishop/chatbot
docker compose exec -T chatwoot-postgres \
  pg_dump -U postgres chatwoot_production > ~/respaldo-chatwoot.sql
```

8. Cuánto cuesta y qué pasa después

Concepto	Costo real
Créditos recibidos de AWS	100 dólares
Servidor	24 dólares al mes
Dominio	0
Certificado de seguridad	0
Modelo de inteligencia artificial	0 (cuota gratuita de Gemini)
Base de datos vectorial y embeddings	0 (software libre en el mismo servidor)

Los créditos alcanzan para unos **cuatro meses**. Dos advertencias importantes:

1. El plan gratuito de AWS **dura seis meses como máximo** y después la cuenta se cierra sola. Conviene respaldar los datos antes.
2. Lightsail **cobra aunque el servidor esté apagado**, porque el disco y la dirección IP siguen reservados. Para dejar de pagar hay que tomar una fotografía del disco (snapshot, que cuesta centavos) y eliminar el servidor.

Si el proyecto debe seguir vivo más tiempo, la misma configuración funciona sin cambios en un servidor de otro proveedor por unos 8 dólares al mes. Solo hay que repetir los pasos de copia y configuración y apuntar el nombre a la nueva IP.

9. Preguntas probables en la defensa

¿Por qué no se usó directamente ChatGPT o un servicio de chatbot comercial? Porque el objetivo era demostrar un sistema que responda únicamente con información verificable de la tienda. Un modelo general inventa precios y promociones. Aquí el modelo solo redacta sobre la evidencia que se le entrega, y esa evidencia sale del catálogo real.

¿Dónde está la inteligencia artificial exactamente? En dos lugares. Ollama, dentro del servidor, convierte textos en vectores para poder buscar por significado y no por palabras exactas. Gemini, en la nube de Google, interpreta la intención del cliente y redacta la respuesta. Lo importante es que Gemini nunca decide qué productos existen.

¿Qué pasa si se cae el servicio de Google? El sistema sigue respondiendo con una interpretación de respaldo que recorre los mismos pasos de búsqueda. Las respuestas son más secas pero siguen siendo correctas.

¿Cómo se garantiza que no invente precios? La respuesta se construye a partir de los documentos recuperados de Qdrant, que provienen del catálogo. La batería de pruebas incluye casos diseñados para detectar invenciones; por ejemplo, al pedir un micrófono (producto que la tienda no vende) el asistente debe reconocer el límite en lugar de ofrecer algo parecido.

¿Por qué siete programas separados y no uno solo? Porque cada uno resuelve bien una cosa y se pueden cambiar de forma independiente. Si mañana aparece una base vectorial mejor que Qdrant, se reemplaza esa pieza sin tocar el resto. Es la misma lógica con la que se diseñan los sistemas en producción.

¿Es seguro? Hay cifrado en tránsito, el cortafuegos expone solo lo indispensable, las bases de datos no son accesibles desde internet, las contraseñas se generaron al azar y se bloqueó el acceso web a los archivos de configuración. No se guardan datos de tarjetas ni información personal sensible.

¿Cuánto costaría operarlo de verdad? Entre 8 y 24 dólares al mes según el proveedor, más cero por el modelo mientras el volumen quepa en la cuota gratuita. Para una tienda pequeña es un costo marginal frente a tener una persona respondiendo las mismas preguntas repetitivas.

10. Limitaciones reconocidas

Declararlas con honestidad fortalece el trabajo:

- Es una demostración académica. No conecta con el inventario real ni con el sistema de pedidos de la tienda.
- No incluye WhatsApp ni seguimiento de envíos.
- La tienda es una copia estática; las páginas que nunca se visitaron en la versión local no existen en el servidor.
- El servidor es de gama básica y comparte procesador con otros clientes, así que las tareas pesadas van lentas.
- La cuota gratuita de Gemini es limitada; con mucho tráfico habría que pagar o cambiar a un modelo propio.

- El asistente responde en español y sobre el catálogo cargado; fuera de ese ámbito deriva a un asesor humano, que es el comportamiento deseado.

11. Glosario

Contenedor (Docker): caja sellada que incluye un programa y todo lo que necesita para funcionar, de modo que se comporta igual en cualquier computadora.

Certificado HTTPS: credencial que permite el candado del navegador y cifra la comunicación para que nadie pueda leerla por el camino.

Proxy inverso: programa que recibe todas las visitas y las reparte al servicio correspondiente. Aquí lo hace Caddy.

Vector o embedding: lista de números que representa el significado de un texto. Dos textos parecidos tienen vectores cercanos, y por eso se puede buscar por sentido y no por palabras exactas.

Base vectorial: base de datos especializada en encontrar los vectores más cercanos a uno dado. Aquí es Qdrant.

RAG: técnica que consiste en buscar primero la información en una fuente propia y después pedirle al modelo que redacte usando solo esa información.

Webhook: dirección de internet que un programa usa para avisar a otro que ocurrió algo. Chatwoot avisa a n8n por este medio cada vez que llega un mensaje.

Swap o memoria de intercambio: espacio del disco que el sistema usa como memoria de emergencia cuando la memoria real se agota.

SSH: forma segura de entrar al servidor desde la computadora y escribir comandos como si se estuviera frente a él.

Snapshot: fotografía completa del disco del servidor, que permite volver a crearlo tal como estaba.